

SmartMask Network VST3

Műszaki Architektúra, Pszichoakusztikus DSP Specifikáció és AI Implementation Blueprint

Célplatform: Windows / macOS Fejlesztő Keretrendszer: C++20 Cél AI Modell: Claude 3.7 / 4 / 5 Dokumentum Verzió: (VST3, AU) / JUCE 8 (Sonnet & Opus) v1.0.0-FINAL

1. A Plugin Lényege és Funkcionális Specifikációja (Tőszavakban)

A **SmartMask Network** egy sávok közötti valós idejű spektrális maszkolás-kezelő VST3/AU audio plugin. A megszokott egyirányú sidechain kompresszorokkal szemben egy globális, hálózatos állapotregisztert használ, amely lehetővé teszi a DAW csatornái közötti intelligens, többirányú frekvenciaterület-megosztást.

Fő Funkciók és Képességek (Core Features):

- Inter-Plugin Shared Memory Registry:** Lock-free, szálbiztos memóriaterület az összes DAW-ban betöltött SmartMask példány folyamatos, alacsony látenciájú kommunikációjához.
- Real-time STFT Analízis:** 2048-pontos Fast Fourier Transform (75%-os overlap-add) a folyamatos frekvenciaspektrum előállításához.
- Pszichoakusztikus Bark-sáv Csoportosítás:** A frekvencia-binek összevonása 24 pszichoakusztikus kritikus sávba (Bark Scale) az emberi hallás biológiai modelljéhez igazodva.
- Dinamikus Prioritási Fa (Dynamic Priority Matrix):** Sávonkénti prioritás-beállítás (1 = Legmagasabb/Vokál, 10 = Legalacsonyabb/Pad). A magasabb prioritású sávok automatikusan mélyítik ki saját frekvenciacsúcsaikat az alacsonyabb prioritású sávokból.
- Adaptive Psychoacoustic Suppression (Null-Phase Spectral Notch Filtering):** Célzott frekvenciacsökkentés csak a maszkolt tartományokban, így elkerülhető a teljes sáv szélességű szivattyúzó (ducking) hatás.
- Automatic Delay Compensation (PDC):** Precíz 2048-mintás látencia-bejelentés a host DAW felé (`setLatencySamples`) a tökéletes sample-accurate szinkronizációért.
- Global Spectral Visualizer GUI:** Vektoros JUCE felület, amely egyetlen ablakban mutatja az összes sáv spektrális aktivitását és a sávok közötti egymásra hatást.

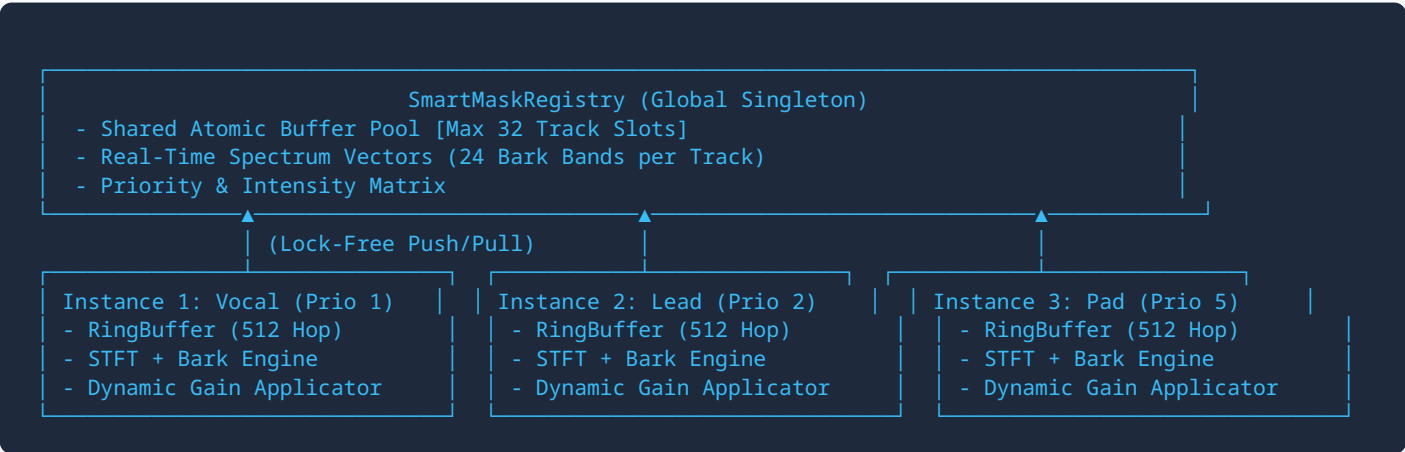
2. Szükséges Előismeretek és Beszerzendő Információk

A fejlesztés megkezdése előtt a kódoló mérnöknek (vagy az AI kódgenerátornak) az alábbi könyvtárakkal, matematikai modellekkel és architektúrákkal kell rendelkeznie:

Kategória	Szükséges Eszköz / Információ	Fejlesztői Cél és Használat
Szoftver Keretrendszer	JUCE 8.x (C++20 kompatibilis mode)	AudioProcessor, AudioProcessorEditor, APVTS, DSP modulok (<code>juce::dsp::FFT</code>).
DSP & Matematika	Eberhard Zwicker Bark Scale formula & ISO 226 Equal-Loudness	A lineáris FFT binek átváltása 24 pszichoakusztikus kritikus sávba: $z = 13 * \arctan(0.00076*f) + 3.5 * \arctan((f/7500)^2)$.
Szálkezelés (Threading)	Lock-free Data Structures (SPSC / MPMC Ring Buffers)	Mutex-mentes adatmegosztás az Audio Thread és a Shared Registry között. Atomic pointer swapping.
Ablakozás & STFT	Hann Window & Overlap-Add (OLA) Rekonstrukció	75%-os ablakfedés (Hop Size: 512 sample 2048-as FFT-nél) a fázi-artefaktumok és kattogások megszüntetésére.
DAW Host IPC	Process-Isolation Specs (Bitwig / FL Studio Sandbox)	Tudnivalók az egyfolyamatos (Single Process) vs multi-process processzor-izolációról (Shared Memory IPC megléte).
SIMD Optimalizáció	<code>juce::FloatVectorOperations</code> / AVX2 / ARM NEON	A 24 Bark-sáv gain-szorzásának vektorizált végrehajtása alacsony CPU-terhelés mellett.

3. A Rendszer Architektúrája és Felépítési Tervrajza

A moduláris felépítés 5 fő komponenst különít el szigorúan definiált felelősségi körökkel:



A Modulok Részletes Szerepe:

- SmartMaskRegistry** : Száلبiztos, lock-free globális állapotkezelő. Nincs mutex zárolás az audio szálon. Helyette `std::atomic<BarkSpectrum*>` mutatókat cserél kettős puffereléssel (Double Buffering).
- SpectralAnalysisEngine** : Bejövő time-domain minták gyűjtése RingBuffer-be, 2048 mintás Hann ablakozás, FFT számítás, majd a lineáris binek összegezése 24 Bark-sávra.
- PsychoacousticCalculator** : Kiszámítja az adott sávra ható maszkolási görbét: ha egy magasabb prioritású sáv energiája meghaladja a küszöböt a *k*-edik Bark-sávban, a választható intenzitásnak megfelelően csökkenti a gain-t.
- SpectralFilterBank** : Az egyesített Bark-sáv gain-értékek visszaskálázása lineáris FFT binkre, az eredeti fázis megőrzése (Zero-phase reconstruction), IFFT és Overlap-Add kimeneti pufferelés.
- SmartMaskAudioProcessor & Editor** : JUCE illesztőfelület, APVTS paraméterek (Priority 1-10, Amount 0-100%, Attack/Release Smooth 5-200ms) és a vizuális kijelző (OpenGL/JUCE 2D Graphics).

4. Kritikus Odafigyelnivalók és Veszélyzónák (Pitfalls & Edge Cases)

Szigorú Szabályok az Audio Szálon (Real-Time Audio Thread Constraints):

- **ZERO Heap Allocation:** A `processBlock` metódusban TILTOTT a `new`, `malloc`, `std::vector::resize` vagy bármilyen dinamikus memóiafogalás. Minden puffert (RingBuffer, FFT work buffers) a `prepareToPlay` szakaszban ELŐRE le kell foglalni!
- **ZERO Mutex / Lock Usage:** TILTOTT a `std::mutex`, `std::lock_guard` vagy `juce::CriticalSection` használata az audio szálon. Az inter-plugin kommunikáció kizárólag lock-free atomic pointer cserével vagy Lock-Free Ring Buffer-rel történhet.
- **Denormal Number Protection:** Mindig kötelező a `juce::ScopedNoDenormals noDenormals;` használata a `processBlock` legelején. A rendkívül kicsi lecsengő float értékek (1e-30) különben 100%-os CPU túske-leállást okozhatnak!

Architektúráis Élek és Speciális Esetek:

- **DAW Sandbox / Process Isolation:** Egyes hostok (pl. Bitwig, FL Studio külön process opció) eltérő folyamatokban futtatják a plugin példányokat. A Singleton memóriacím ilyenkor nem közös! Megoldás: C++ `boost::interprocess` vagy OS-native Named Shared Memory (Win: `CreateFileMapping`, POSIX: `shm_open`) fallback mechanizmus implementálása.
- **Phase Alignment & OLA Reconstruction:** Az IFFT utáni rekonstrukciónál az eredeti phase információt pontosan meg kell tartani, és a Hann ablak 75%-os átfedésének szummatív nyereségét (Gain Normalization Factor = 0.375) korrigálni kell a 0 dB-es transzparens áteresztéshez.
- **PDC (Plugin Delay Compensation):** Az FFT ablakozás 2048 mintás látenciát okoz. Ezt a `prepareToPlay`-ben a `setLatencySamples(2048)` meghívásával tudatni kell a DAW-val. Ennek hiányában a sávok elcsúsznak egymáshoz képest!
- **Dangling Track Registration:** Amikor a felhasználó töröl egy sávot vagy plugint a DAW-ban, a `destructor`-nak biztonságosan, atomic módon ki kell regisztrálnia a sávot a `SmartMaskRegistry`-ből, megakadályozva a memóriaszemét vagy elárvult mutatók olvasását.

5. A Programozás Lépései (Step-by-Step AI Implementation Roadmap)

Az alábbi ütemterv kifejezetten úgy lett szerkesztve, hogy modulárisan átadható legyen **Claude 3.7 / 4 / 5 (Sonnet vagy Opus)** modelleknek. Az egyes fázisok egymásra épülnek.

FÁZIS 1: A Lock-Free Állapotregiszter (Shared Registry) Megírása

- **Feladat:** Hozz létre egy `SmartMaskRegistry.h` fejléct.
- **Specifikáció:** Fix 32 slotos tömb. Minden slot tartalmazza: `trackId` (int), `priority` (`std::atomic<int>`), `amount` (`std::atomic<float>`), és egy dupla puffert a 24 Bark-sáv energiáihoz (`std::atomic<BarkData*>`).
- **Kritérium:** Írj Catch2 unit tesztet, ami 16 párhuzamos szálon írja és olvassa a regisztert akadálymentesen.

```
// Fázis 1 Struktúra Minta (AI Prompting Alap):
struct Alignas(64) BarkSpectrum {
    std::array<float, 24> bandEnergies{};
};

class SmartMaskRegistry {
public:
    static SmartMaskRegistry& getInstance();
    int registerTrack();
    void unregisterTrack(int slotIndex);
    void updateSpectrum(int slotIndex, const std::array<float, 24>& incomingBands);
    void getGlobalMask(int currentSlotIndex, int currentPriority, std::array<float, 24>& outMaskGains);
private:
    std::array<TrackSlot, 32> slots;
};
```

FÁZIS 2: STFT & Bark-Scale Pszichoakusztikus DSP Engine

- **Feladat:** Hozz létre egy `SpectralEngine.cpp/h` osztályt.
- **Specifikáció:** 1. RingBuffer használata 512 mintás lépésközhöz (Hop Size).
2. 2048-as Hann ablakozás futtatása a `juce::dsp::FFT`-re.
3. A 1024 lineáris bin áttanszformálása 24 Bark-sávba egy előre kiszámított index-leképező mátrix (Lookup Table) segítségével.
4. Simítás (Smoothing) alkalmazása az energiákra: Attack (10ms) és Release (50ms) szűrő a frekvenciátüskék felpuhítására.

FÁZIS 3: A Prioritási Mátrix és Dinamikus Szűrő Számítása

- **Feladat:** Hozz létre egy `MaskingProcessor.cpp/h` osztályt.
- **Specifikáció:** 1. A `SmartMaskRegistry`-ből lekérdezi az összes nálunk magasabb prioritású sáv 24 Bark energiáját.
2. Kiszámítja a Bark-sávonkénti csillapítási tényezőt (0.0f - 1.0f).
3. A 24 Bark gain értéket visszainterpólálja 1024 lineáris bin szorzóvá.
4. A korrigált komplex FFT spektrumot IFFT-zi, majd az Overlap-Add módszerrel összefűzi a kimeneti pufferbe.

FÁZIS 4: JUCE AudioProcessor & APVTS Integráció

- **Feladat:** Készítsd el a `PluginProcessor.cpp/h` osztályt.
- **Specifikáció:** 1. APVTS paraméterek bekötése: `Priority` (Choice: 1-10), `Amount` (Float: 0-100%), `Attack`, `Release`, `Smoothing`.
2. Hívd meg a `setLatencySamples(2048)` -at a konstruktorban / `prepareToPlay`-ben.
3. Az Audio szálon történő futtatás ellenőrzése Real-Time teszterrel.

FÁZIS 5: Vektoros GUI és Spektrális Hálózati Kijelző

- **Feladat:** Fejleszd ki a `PluginEditor.cpp/h` vizuális felületet.
- **Specifikáció:** 1. Hozz létre egy élő spektrum-megjelenítőt (Spectrum Visualizer Component).
2. Megjeleníti az összes aktív sávot színes görbéként.
3. Drag-and-Drop prioritás-átrendező lista.
4. JUCE Timer (30 Hz frissítés) segítségével rajzold újra a képernyőt (soha ne az audio szálról!).

FÁZIS 6: SIMD Optimalizáció & Terheléses Tesztelés

- **Feladat:** Kódoptimalizálás és profilozás.
- **Specifikáció:** 1. Használd `juce::FloatVectorOperations::multiply` -t a gain-szorításokhoz.
2. Futtass 32 aktív plugin példányt Ableton Live-ban és Reaper-ben 96 kHz-en.
3. CPU terhelés cél: példányonként kevesebb mint 0.3% egy modern Apple M1/M2 vagy Intel i7 processzoron.

6. Prompting Matriks AI Asszisztensekhez (Claude Instructions)

Hogyan add át ezt a dokumentumot a Claude Sonnet 3.7 / Opus 4 / Sonnet 5-nek:

Ha a kódoltatást elindítod, másold be az alábbi Promptot a Claude-nak a PDF kíséretében:

"Szia! Mellékelten átadom a SmartMask Network VST3 plugin teljes műszaki specifikációs dokumentumát. A feladatod, hogy C++20 és JUCE 8 keretrendszerben készítsd el a szoftvert. Kérlek, szigorúan kövesd az 5. Fejezetben leírt programozási lépéseket (Fázis 1-től Fázis 6-ig). Különösen ügyelj a 4. Fejezetben kiemelt Real-Time szálbiztonsági szabályokra (Zero allocation and Zero locks in processBlock). Első lépésként írd meg a Fázis 1-ben kért Lock-Free SmartMaskRegistry C++ modul teljes kódját!"